Online Graph Embedding in Star Graphs

J. Dallot D. Melnyk M. Pacut S. Schmid

TU Berlin

ICDCS 2026

Outline

Online Graph Embedding

Related Work & Contributions

The Deterministic Case

The Randomized Case

Online Graph Embedding

Introducing Online Graph Embedding

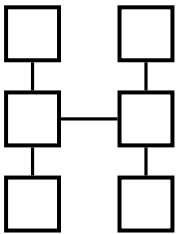
Graph embedding involves two graphs:

- the **guest graph**: the graph to be embedded, and
- the **host graph**: the graph into which the guest is embedded.

Introducing Online Graph Embedding

Graph embedding involves two graphs:

- the **guest graph**: the graph to be embedded, and
- the **host graph**: the graph into which the guest is embedded.

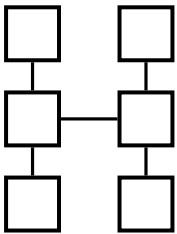


Host graph H

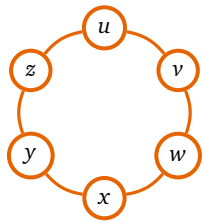
Introducing Online Graph Embedding

Graph embedding involves two graphs:

- the **guest graph**: the graph to be embedded, and
- the **host graph**: the graph into which the guest is embedded.



Host graph H

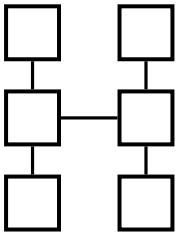


Guest graph G

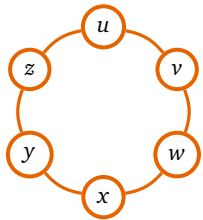
Introducing Online Graph Embedding

Graph embedding involves two graphs:

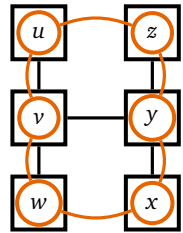
- the **guest graph**: the graph to be embedded, and
- the **host graph**: the graph into which the guest is embedded.



Host graph H



Guest graph G



An embedding of G in H

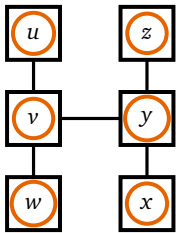
This embedding has a total cost of 12.

Graph Embedding: the Online Variant

In the online variant, the guest graph is **revealed edge by edge** and not all at once.

Graph Embedding: the Online Variant

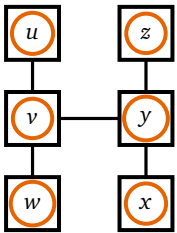
In the online variant, the guest graph is **revealed edge by edge** and not all at once.



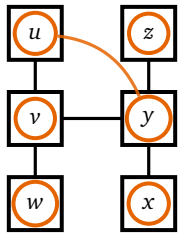
Initial embedding

Graph Embedding: the Online Variant

In the online variant, the guest graph is **revealed edge by edge** and not all at once.



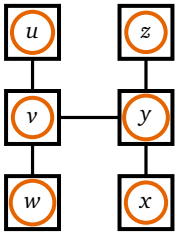
Initial embedding



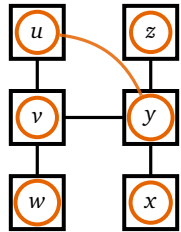
Edge $\{u,y\}$ revealed (cost 2)

Graph Embedding: the Online Variant

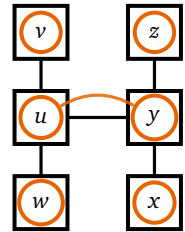
In the online variant, the guest graph is **revealed edge by edge** and not all at once.



Initial embedding



Edge $\{u,y\}$ revealed (cost 2)



Swap $u \leftrightarrow v$ (cost 1). Now serving $\{u,y\}$ would cost 1

Online Graph Embedding: Problem Statement

Input.

- A **host graph** $H = (V_H, E_H)$ with distance d_H
- A **guest graph** $G = (V_G, E_G)$ such that $|V_G| = |V_H|$
- An initial bijection $\phi_0 : V_G \to V_H$
- A sequence of edge requests $\sigma = e_1, e_2, \ldots,$ $e_i \in E_G$, revealed **one by one**

Online Graph Embedding: Problem Statement

Input.

- A **host graph** $H = (V_H, E_H)$ with distance d_H
- A **guest graph** $G = (V_G, E_G)$ such that $|V_G| = |V_H|$
- An initial bijection $\phi_0 : V_G \to V_H$
- A sequence of edge requests $\sigma = e_1, e_2, \ldots,$ $e_i \in E_G$, revealed **one by one**

Per request $e_i = \{u, v\}$**.** The algorithm may first **reconfigure** ϕ (swap two guest nodes), then must **serve** e_i:

$$\text{cost}(e_i) = \underbrace{d_\tau(\phi_{i-1}, \phi_i)}_{\text{reconfig.}} + \underbrace{d_H(\phi_i(u), \phi_i(v))}_{\text{serving}}$$

where d_τ counts the number of swaps.

Online Graph Embedding: Problem Statement

Input.

- A **host graph** $H = (V_H, E_H)$ with distance d_H
- A **guest graph** $G = (V_G, E_G)$ such that $|V_G| = |V_H|$
- An initial bijection $\phi_0 : V_G \to V_H$
- A sequence of edge requests $\sigma = e_1, e_2, \ldots$, $e_i \in E_G$, revealed **one by one**

Goal. Minimise $\sum_i \text{cost}(e_i)$ *online*.

Per request $e_i = \{u, v\}$**.** The algorithm may first **reconfigure** ϕ (swap two guest nodes), then must **serve** e_i:

$$\text{cost}(e_i) = \underbrace{d_\tau(\phi_{i-1}, \phi_i)}_{\text{reconfig.}} + \underbrace{d_H(\phi_i(u), \phi_i(v))}_{\text{serving}}$$

where d_τ counts the number of swaps.

Online Graph Embedding: Problem Statement

Input.

- A **host graph** $H = (V_H, E_H)$ with distance d_H
- A **guest graph** $G = (V_G, E_G)$ such that $|V_G| = |V_H|$
- An initial bijection $\phi_0 : V_G \to V_H$
- A sequence of edge requests $\sigma = e_1, e_2, \ldots$, $e_i \in E_G$, revealed **one by one**

Per request $e_i = \{u, v\}$**.** The algorithm may first **reconfigure** ϕ (swap two guest nodes), then must **serve** e_i:

$$\text{cost}(e_i) = \underbrace{d_\tau(\phi_{i-1}, \phi_i)}_{\text{reconfig.}} + \underbrace{d_H(\phi_i(u), \phi_i(v))}_{\text{serving}}$$

where d_τ counts the number of swaps.

Goal. Minimise $\sum_i \text{cost}(e_i)$ *online*.

Competitive analysis

Algorithm Alg is c**-competitive** if for every request sequence σ:

$$\mathsf{Alg}(\sigma) \le c \cdot \mathsf{Opt}(\sigma)$$

where Opt is the best *offline* algorithm.

Outline

Online Graph Embedding

Related Work & Contributions

The Deterministic Case

The Randomized Case

Outline

Online Graph Embedding

Related Work & Contributions

Related Work & Contributions

The Randomized Case

Related Work

Single-node requests

Classical online data-structure problems where each request touches *one* node:

- **List access problem** [Sleator and Tarjan 1985]: minimise access + transposition cost on a linked list.
- **Self-adjusting BSTs** (splay trees) [Sleator and Tarjan 1985]: restructure a search tree after each query.
- **VM migration** [Kamali and López-Ortiz 2013]: move a single virtual machine to reduce access cost.

$\Longrightarrow$ *Requests involve a single node; interaction between pairs is not considered.*

Pairwise requests (our setting)

Problems where each request involves a *pair* of nodes:

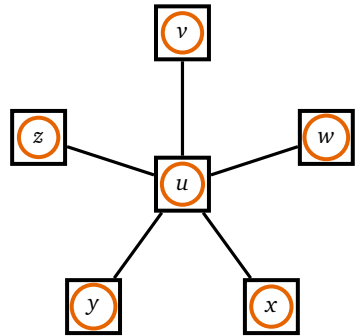
- **Dynamic graph partitioning** [Avin+ 2020; Henzinger+ 2019; Henzinger+ 2021]: intra-cluster cost 0, inter-cluster cost 1.
- **Dynamic min. linear arrangement** [Olver + 2018; Bienkowski and Even 2024]: nodes on a line; cost = distance between requested pair.
- **Online graph embedding** (this work): general host graph; reconfiguration by node swaps.

$\Longrightarrow$ *Pairwise interactions require tracking pairs of nodes simultaneously. Strictly harder.*

Contributions

For the online embedding problem in star graphs, we derive **optimal online algorithms** for both the deterministic and randomized setting.

	Lower bound	**Algorithm**
Deterministic	3/2	3/2
Randomized	11/9	11/9



$\Longrightarrow$ We present the first optimal algorithm for the fundamental case of a node and its neighborhood.

Outline

Online Graph Embedding

Related Work & Contributions

The Deterministic Case

The Randomized Case

Outline

Online Graph Embedding

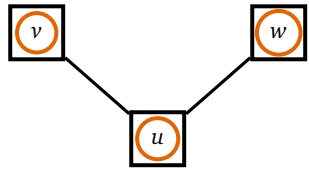
Related Work & Contributions

The Deterministic Case

The Randomized Case

Deterministic Lower Bound: 1.5

Setup. Fix any deterministic algorithm Alg and three guest nodes u, v, w on a 3-node star host.

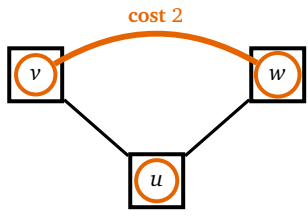


Deterministic Lower Bound: 1.5

Setup. Fix any deterministic algorithm Alg and three guest nodes u, v, w on a 3-node star host.

Adversarial Request Sequence σ. Always request the *two nodes not at* Alg*'s center*.

$\Longrightarrow$ Alg pays **2** per request, every time.



Deterministic Lower Bound: 1.5

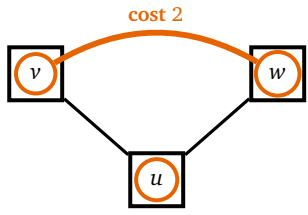
Setup. Fix any deterministic algorithm Alg and three guest nodes u, v, w on a 3-node star host.

Adversarial Request Sequence σ. Always request the *two nodes not at* Alg*'s center*.

$\Longrightarrow$ Alg pays **2** per request, every time.

Offline optimum. One node appears in $\geq \frac{2}{3}|\sigma|$ requests, say it is u. Define Opt that places u at the center and do not move. Then:

$$\mathsf{Opt}(\sigma) \leq \underbrace{1 \cdot \tfrac{2}{3}|\sigma|}_{\text{involving center}} + \underbrace{2 \cdot \tfrac{1}{3}|\sigma|}_{\text{others}} = \tfrac{4}{3}|\sigma|$$

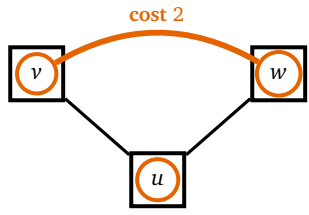


Deterministic Lower Bound: 1.5

Setup. Fix any deterministic algorithm Alg and three guest nodes u, v, w on a 3-node star host.

Adversarial Request Sequence σ. Always request the *two nodes not at* Alg*'s center*.

$\Longrightarrow$ Alg pays **2** per request, every time.



Offline optimum. One node appears in $\geq \frac{2}{3}|\sigma|$ requests, say it is u. Define Opt that places u at the center and do not move. Then:

$$\mathsf{Opt}(\sigma) \leq \underbrace{1 \cdot \tfrac{2}{3}|\sigma|}_{\text{involving center}} + \underbrace{2 \cdot \tfrac{1}{3}|\sigma|}_{\text{others}} = \tfrac{4}{3}|\sigma|$$

Ratio. $\dfrac{\mathsf{Alg}(\sigma)}{\mathsf{Opt}(\sigma)} \geq \dfrac{2|\sigma|}{\frac{4}{3}|\sigma|} = \mathbf{\dfrac{3}{2}}$

PivotTracking: Our Deterministic Algorithm

Main Idea. Maintain a **candidate set** C of guest nodes that Opt might be keeping at the center.

- If the requested edge intersects C, migrate a requested node to the center (**intersection behavior**).
- Else, grow the candidate set (**union behavior**).

PivotTracking: Our Deterministic Algorithm

Main Idea. Maintain a **candidate set** C of guest nodes that Opt might be keeping at the center.

- If the requested edge intersects C, migrate a requested node to the center (**intersection behavior**).
- Else, grow the candidate set (**union behavior**).

PivotTracking

Init: $x_0 \leftarrow$ initial center, $C \leftarrow \{x_0\}$

On request $\sigma_t = \{x_1, x_2\}$:

intersection

- **if** $C \cap \{x_1, x_2\} \neq \emptyset$ **then**
  - $C \leftarrow C \cap \{x_1, x_2\}$
  - **if** no node of C at center **then**
    - migrate a node of C to center

union

- **else**
  - $C \leftarrow C \cup \{x_1, x_2\}$

serve σ_t

A Few Helpful Definitions

Phase and pivot

A **phase** of an algorithm is a maximal run of requests during which its central node does not change. The central node during a phase is called the **pivot**.

Opt*

The optimal offline algorithm that minimizes the *reversed lexicographic order* on phase lengths, i.e. it delays phase changes as long as possible.

Cheap and expensive requests

A request σ_i is **cheap** if Opt* pays 1 for it, and **expensive** if Opt* pays 2.

The Tracking Lemma

Tracking Lemma

Let σ_i be any request and C the candidate set. Then:

Alg uses **intersection** $\iff$ σ_i is cheap Alg uses **union** $\iff$ σ_i is expansive

Intersection behavior

$C \leftarrow C \cap \sigma_i$

Migrate a node from C to center if needed.

Union behavior

$C \leftarrow C \cup \sigma_i$

No migration.

Proof of the Tracking Lemma (1/2): Cheap ⇒ Intersection

We prove the claim by induction on the successive requests.
Claim. If σ_i is cheap, then Alg uses **intersection**.

Recall. σ_i is cheap means Opt^* pays 1, so the pivot x of the current phase is in σ_i.

Proof of the Tracking Lemma (1/2): Cheap ⇒ Intersection

We prove the claim by induction on the successive requests.
Claim. If σ_i is cheap, then Alg uses **intersection**.

Recall. σ_i is cheap means Opt^* pays 1, so the pivot x of the current phase is in σ_i.
Step 1. x is in C at the start of the phase.

- *Base case (first phase):* x was inserted in C at initialization.
- *Later phase:* by induction: a phase starts with an expensive request containing x so Alg applied **union** $\implies$ x entered C.

Proof of the Tracking Lemma (1/2): Cheap ⇒ Intersection

We prove the claim by induction on the successive requests.
Claim. If σ_i is cheap, then Alg uses **intersection**.

Recall. σ_i is cheap means Opt^* pays 1, so the pivot x of the current phase is in σ_i.
Step 1. x is in C at the start of the phase.

- *Base case (first phase):* x was inserted in C at initialization.
- *Later phase:* by induction: a phase starts with an expensive request containing x so Alg applied **union** $\implies$ x entered C.

Step 2. x stays in C from the phase start up to σ_i.

- **Expensive** requests only *add* nodes to C: x not removed.
- **Cheap** requests intersect C with σ_j, which always contains x (since x is the pivot and Opt^* pays 1): x stays.

Proof of the Tracking Lemma (1/2): Cheap ⇒ Intersection

We prove the claim by induction on the successive requests.
Claim. If σ_i is cheap, then Alg uses **intersection**.

Recall. σ_i is cheap means Opt* pays 1, so the pivot x of the current phase is in σ_i.
Step 1. x is in C at the start of the phase.

- *Base case (first phase):* x was inserted in C at initialization.
- *Later phase:* by induction: a phase starts with an expensive request containing x so Alg applied **union** $\implies$ x entered C.

Step 2. x stays in C from the phase start up to σ_i.

- **Expensive** requests only *add* nodes to C: x not removed.
- **Cheap** requests intersect C with σ_j, which always contains x (since x is the pivot and Opt* pays 1): x stays.

Conclusion.

$$x \in C \quad \text{and} \quad x \in \sigma_i \quad \implies \quad C \cap \sigma_i \neq \emptyset \quad \implies \quad \text{Alg uses } \textbf{intersection.} \qquad \square$$

Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection.

Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

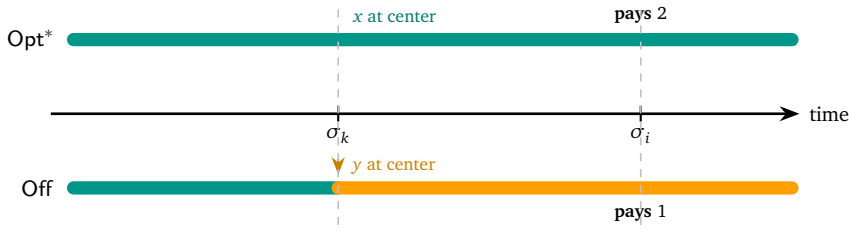
Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection. Then $\exists y \in C \cap \sigma_i$. Let σ_k be the earliest request after which y stayed in C continuously.

Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection.

Then $\exists y \in C \cap \sigma_i$. Let σ_k be the earliest request after which y stayed in C continuously.

Construct Off**:** act like Opt*, but *migrate y to center just before σ_k* and keep it there through σ_i.



Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection.

Then $\exists y \in C \cap \sigma_i$. Let σ_k be the earliest request after which y stayed in C continuously.

Off **saves 1 on σ_i.**

- $y \in \sigma_i$ but $x \notin \sigma_i$ (since σ_i is expensive, the pivot x is *not* in σ_i).
- Off has y at center $\implies$ pays 1. Opt* has x at center $\implies$ pays 2.

The migration cost of y is *already covered*: by induction, σ_k was expensive, so Opt* also paid a migration cost there.

Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection.

Then $\exists y \in C \cap \sigma_i$. Let σ_k be the earliest request after which y stayed in C continuously.

After σ_i: two cases, both give a contradiction:

- x **never appears again in the phase:** Off pays $\leq$ Opt* on all remaining requests, but already saved 1 on σ_i ⇒ Off strictly better than Opt*. **Contradiction.**
- x **appears later at** $\sigma_{k'}$**:** Let Off migrate x at $\sigma_{k'}$ (cost +1), losing the saving. Total cost = Opt*, but the phase of x at center is *shorter* ⇒ Off has smaller reversed lexicographic order than Opt*. **Contradiction.**

Proof of the Tracking Lemma (2/2): Expensive ⇒ Union

Claim. If σ_i is expensive, Alg uses **union**. Proof by **contradiction**: assume Alg uses intersection.

Then $\exists y \in C \cap \sigma_i$. Let σ_k be the earliest request after which y stayed in C continuously.

After σ_i: two cases, both give a contradiction:

- x **never appears again in the phase:** Off pays $\leq$ Opt* on all remaining requests, but already saved 1 on σ_i ⇒ Off strictly better than Opt*. **Contradiction.**
- x **appears later at $\sigma_{k'}$:** Let Off migrate x at $\sigma_{k'}$ (cost +1), losing the saving. Total cost = Opt*, but the phase of x at center is *shorter* ⇒ Off has smaller reversed lexicographic order than Opt*. **Contradiction.**

Conclusion. Both cases contradict the definition of Opt*.

Intersection on expensive request is impossible $\Longrightarrow$ Alg uses **union.** $\square$

Proving 1.5-Competitiveness: Block Decomposition

Main idea. Partition σ into *blocks*, analyse each block separately, and show the ratio is ≤ 1.5 on every block.

Using the tracking lemma, label each request:

$$\mathscr{E} = \text{expensive request}, \qquad \Gamma = \text{cheap request}.$$

Block decomposition of σ:

$$\sigma = \underbrace{\Gamma^*}_{\text{prefix}} \underbrace{\left(\mathscr{E}^+\Gamma^+\right)^*}_{\text{block}} \underbrace{\mathscr{E}^*}_{\text{suffix}}$$

A **block** $B = \mathscr{E}^+\Gamma^+$: one or more expensive requests followed by one or more cheap requests.

Proving 1.5-Competitiveness: Block Decomposition

$$\sigma = \Gamma^* \left(\mathscr{E}^+ \Gamma^+ \right)^* \mathscr{E}^*$$

Prefix and suffix are easy.

- **Prefix** Γ^*: all requests are cheap. By tracking lemma, Alg uses intersection: it never pays more than Opt^*. Ratio ≤ 1.
- **Suffix** $\mathscr{E}^*$: all requests are expensive. Alg uses union: no migration, pays 2 per request. Opt^* also pays ≥ 2 (migration + serve or just 2). Ratio ≤ 1.

It remains to analyse each **block** $B = \mathscr{E}^+ \Gamma^+$.

Proving 1.5-Competitiveness: Block Decomposition

Analysis of a block B with ε expensive and γ cheap requests.

Opt^* pays exactly:

$$\mathsf{Opt}^*(B) = 2\varepsilon + \gamma$$

Alg on the γ cheap requests (intersection behavior): C shrinks at most **twice** during a block, each shrink costs at most 1 extra.

$$\mathsf{Alg}(B) \;\le\; 2\varepsilon + \gamma + \min(2, \gamma)$$

Competitive ratio on B:

$$\frac{\mathsf{Alg}(B)}{\mathsf{Opt}^*(B)} \;\le\; \frac{2\varepsilon + \gamma + \min(2,\gamma)}{2\varepsilon + \gamma} \;\le\; 1 + \frac{2}{2 \cdot 1 + 2} \;=\; \tfrac{3}{2} \qquad \square$$

Outline

Online Graph Embedding

Related Work & Contributions

The Deterministic Case

The Randomized Case

Outline

Online Graph Embedding

Related Work & Contributions

The Randomized Case

The Randomized Case

Random PivotTracking

Same structure as PivotTracking, with **two randomized tweaks**:

Random PivotTracking

Init: $x_0 \leftarrow$ initial center, $C \leftarrow \{x_0\}$

On request $\sigma_t = \{x_1, x_2\}$:

intersection

if $C \cap \{x_1, x_2\} \neq \emptyset$ **then**

$C \leftarrow C \cap \{x_1, x_2\}$

if no node of C at center **then**

migrate a node of C to center, **uniformly at random**

union

else

$C \leftarrow C \cup \{x_1, x_2\}$

with equal prob. $\frac{1}{3}$: do nothing, migrate x_1, or migrate x_2

serve σ_t

We prove that Random PivotTracking has a competitive ratio of $11/9 \approx 1.22$.
The proof uses a slightly more complicated block partition of the input sequence.

Randomized Lower Bound: 11/9

The adversary constructs the sequence as **pairs** of requests. Each pair has a **pivot** : the node the offline algorithm Off keeps at the center.

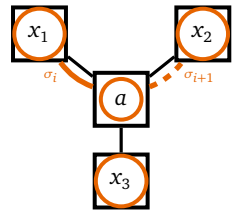
Pattern 1 (prob. p): pivot a appears in *both* requests.

$$(\{a, x_1\},\ \{a, x_2\})$$

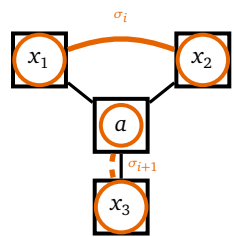
Pattern 2 (prob. $1-p$): pivot a (from prev. pair) appears only in the *second* request.

$$(\{x_1, x_2\},\ \{a, x_3\})$$

Pattern 1 (pivot a in both)



Pattern 2 (pivot a in 2nd only)



Randomized Lower Bound: 11/9

The adversary constructs the sequence as **pairs** of requests. Each pair has a **pivot** : the node the offline algorithm Off keeps at the center.

Pattern 1 (prob. p): pivot a appears in *both* requests.

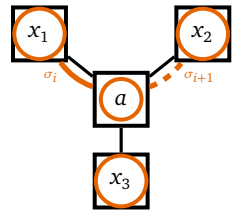
$$(\{a, x_1\},\ \{a, x_2\})$$

Pattern 2 (prob. $1-p$): pivot a (from prev. pair) appears only in the *second* request.

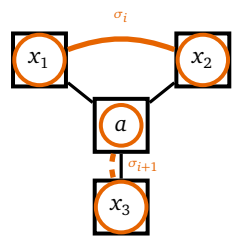
$$(\{x_1, x_2\},\ \{a, x_3\})$$

Off migrates a to center before each pair $\Rightarrow$ pays exactly **3** per pair.

Pattern 1 (pivot a in both)



Pattern 2 (pivot a in 2nd only)



Randomized Lower Bound: 11/9

The adversary constructs the sequence as **pairs** of requests. Each pair has a **pivot** : the node the offline algorithm Off keeps at the center.

Pattern 1 (prob. p): pivot a appears in *both* requests.

$$(\{a, x_1\},\ \{a, x_2\})$$

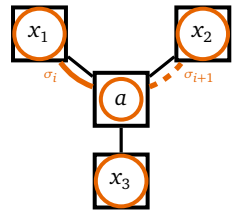
Pattern 2 (prob. $1-p$): pivot a (from prev. pair) appears only in the *second* request.

$$(\{x_1, x_2\},\ \{a, x_3\})$$

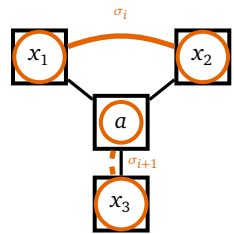
Off migrates a to center before each pair $\Rightarrow$ pays exactly **3** per pair.

Main Idea. Set $p = \frac{2}{3}$: any deterministic Alg pays $\geq \frac{11}{3}$ per pair in expectation.

Pattern 1 (pivot a in both)

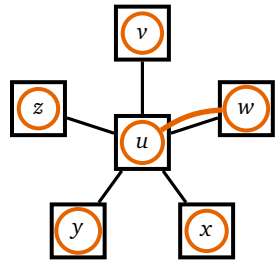


Pattern 2 (pivot a in 2nd only)



Conclusion

- We introduced **online graph embedding** in star graphs: guest edges revealed one by one, algorithm serves and reconfigures.
- Proved **tight** competitive ratios:
  - 3/2 for deterministic algorithms
  - 11/9 for randomized algorithms



Open questions

- Extend to **more complex host graphs** (paths, trees, general topologies).
- Online embedding with **weighted** edges or **capacitated** hosts.

Thank you for your attention!

J. Dallot D. Melnyk M. Pacut S. Schmid

TU Berlin — ICDCS 2026

	Lower bound	Algorithm
Deterministic	3/2	3/2
Randomized	11/9	11/9

References I

Avin, Chen, Marcin Bienkowski, Andreas Loukas, Maciej Pacut, and Stefan Schmid (2020). “Dynamic Balanced Graph Partitioning”. In: *SIAM J. Discret. Math.* 34.3, pp. 1791–1812.

Bienkowski, Marcin and Guy Even (2024). “An Improved Approximation Algorithm for Dynamic Minimum Linear Arrangement”. In: *41st International Symposium on Theoretical Aspects of Computer Science, STACS 2024, March 12-14, 2024, Clermont-Ferrand, France*, 15:1–15:19.

Henzinger, Monika, Stefan Neumann, Harald Räcke, and Stefan Schmid (2021). “Tight Bounds for Online Graph Partitioning”. In: *Proceedings of the 2021 ACM-SIAM Symposium on Discrete Algorithms, SODA 2021, Virtual Conference, January 10 - 13, 2021*, pp. 2799–2818.

Henzinger, Monika, Stefan Neumann, and Stefan Schmid (2019). “Efficient Distributed Workload (Re-)Embedding”. In: *Proc. ACM Meas. Anal. Comput. Syst.* 3.1, 13:1–13:38.

Kamali, Shahin and Alejandro López-Ortiz (2013). “A Survey of Algorithms and Models for List Update”. In: vol. 8066. Lecture Notes in Computer Science. Springer, pp. 251–266.

Olver, Neil, Kirk Pruhs, Kevin Schewior, René Sitters, and Leen Stougie (2018). “The Itinerant List Update Problem”. In: *Approximation and Online Algorithms*. Ed. by Leah Epstein and Thomas Erlebach. Springer International Publishing.

References II

Sleator, Daniel Dominic and Robert Endre Tarjan (1985). “Amortized Efficiency of List Update and Paging Rules”. In: *Commun. ACM* 28.2, pp. 202–208.